Object Lens

System design and software quality

Private technical companion
Edition 1.0
2 November 2026

Object Lens is a small local browser application for exploring object detection in a photograph. The user can choose a photo, inspect the model's predictions, adjust the minimum displayed score and save an annotated image. A sample-photo action provides a quick way to try the same workflow.

Source: docs/DESIGN.md

This companion explains what each part does, why it exists and how the application is checked. It is written as a learning reference for a validation engineer moving into software automation. The explanations follow one image through the application and one requirement through its automated test.

The documented source version, execution records and release assessment identify the scope of the conclusions. A working interface and passing application tests do not establish general object-recognition accuracy.

Document control	Value
Prepared for	Tan Le Jie
Application source	781d9b75db1a3ebc9a200c238038b64aa108564a
Evidence version	docs/evidence/extension-verification.json
Scope	Local Windows and Chrome learning and demonstration workflow

Contents

Scope and user workflow **3**
Objectives and completion 4
Technology and runtime architecture **4**
Server and worker boundaries 6
Image processing and result behavior **6**
From file bytes to predictions 6
Current image ownership 7
Bounded recovery 8
Filtering samples and PNG output **8**
Minimum model score 8
Try sample 8
Save result 9
Testing with Python pytest and Selenium **10**
The two commands 10
Controlled checks and actual inference 10
A worked requirement 11
Requirements and test design **12**
Choosing useful cases 13
Failure investigation **13**
A real narrow viewport defect 14
A deliberate stale-result fault 14
A prediction mistake is a different kind of finding 15
An unexpected file in a download check 15
Verification and release assessment **15**
Release decision 16
Running and maintaining the project **17**
Use the application 17
Run checks 17
Make a bounded change 17
AI assistance and honest demonstration 18
Terms and source references **18**

Reading this companion

Read Scope and user workflow first, then Technology and runtime architecture. Follow Image processing and result behavior to understand the app. Testing with Python pytest and Selenium explains the separate automation path. Verification and release assessment records the observed outcomes.

Architecture figures identify components and their responsibilities. Sequence figures show message order from top to bottom; they are not measured timelines. Actual screenshots are labelled as captures. Controlled responses, deliberately injected faults and real model observations are identified separately.

Scope and user workflow

The intended user is a developer, quality engineer or learner who wants to try a pretrained detector and understand its predictions. Object Lens focuses on one photo at a time, one model and Chrome on Windows. Its small scope keeps the application and its automation understandable.

The main workflow is to choose an image or try the sample, wait for inference, inspect the boxes and matching result rows, adjust the minimum score and save the visible annotations as PNG. Clear removes the current image and its results. Replacing a photo starts a new image workflow.

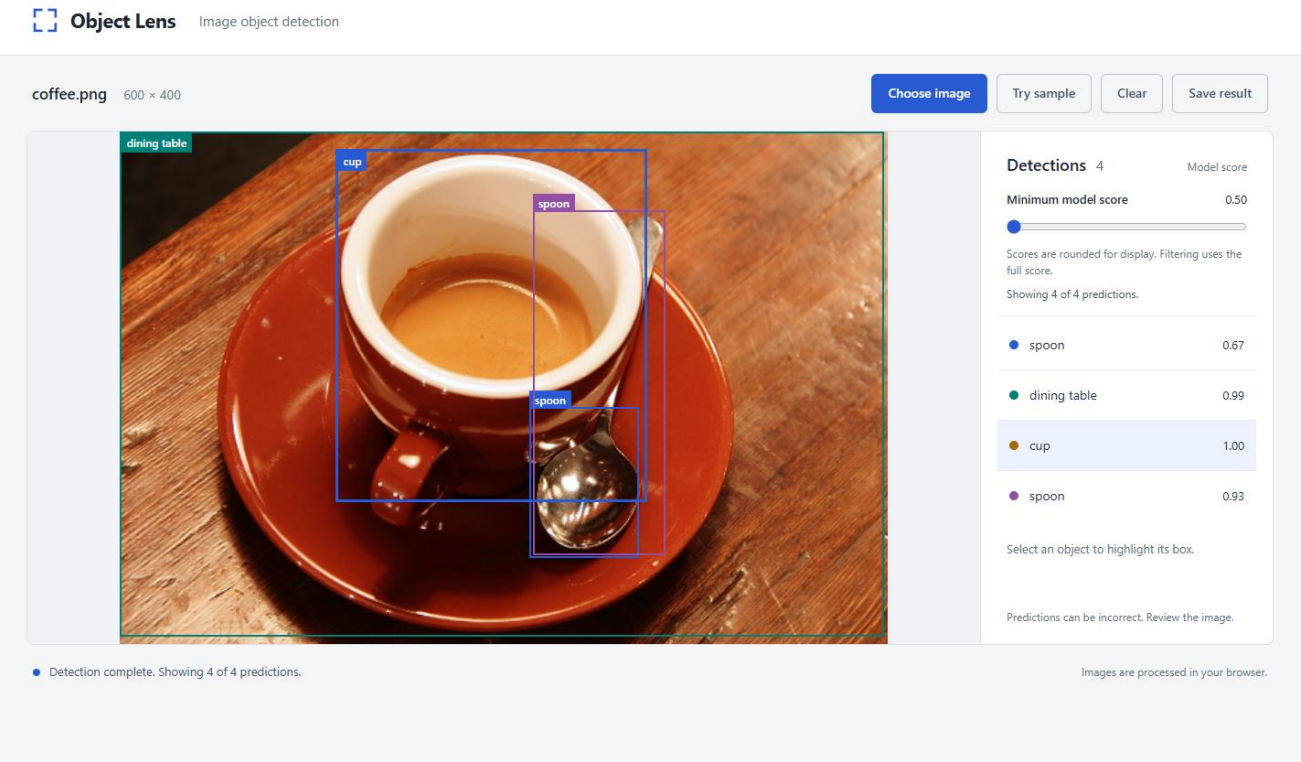


Figure 1 Object Lens after real inference on the public coffee sample. The selected prediction links a result row to its box.

Table 1 Product boundaries

Area	Included	Limit
Input	One decodable JPEG or PNG	At most 10 MiB and 16,000,000 pixels
Detection	One pretrained DETR model in the browser	Model classes and recognition limitations remain those of the chosen model
Interaction	Prediction selection, score filter, sample action, replace and Clear	No saved image history or batch workflow
Output	PNG with the current visible boxes, labels and scores	No report strip, cloud upload or accuracy certification
Execution	Local Windows application and headless Chrome tests	No hosted deployment, iOS or Safari qualification

Accounts, databases, camera/video processing, model training, a model picker and additional AI services are outside this version. The purpose is to explore a working AI-enabled interface and software test automation without maintaining a larger service.

Objectives and completion

The first objective is a usable path from an image to inspectable predictions. The second is to translate observable requirements into acceptance criteria and useful automated checks. The third is to retain enough evidence to investigate failures and make a limited, reasoned release decision.

Completion requires the agreed features to work together, relevant tests and UI checks to pass, an actual PNG export to match the visible predictions, and this document to describe the final source. Source publication and online deployment remain separate decisions.

Back to contents

Technology and runtime architecture

The web application and its automated tests are two different programs. Opening the application lets a person choose and inspect photos. Running pytest asks Python to execute checks against isolated application instances. The detector belongs to the application path; pytest and Selenium belong to the checking path.

Table 2 Technology responsibilities

Component	What it does	Why it is used here
HTML and CSS	Define the page, controls and presentation	A single page can use native browser features without a frontend framework
JavaScript	Validates input and coordinates image, prediction, filter and export state	The interface and model runtime execute in the browser
Web Worker	Runs model loading and inference away from the page's main thread	The page can respond to controls while model work is running
Transformers.js	Loads the selected model and calls its object-detection pipeline	Supplies a mature pretrained-model interface
ONNX Runtime Web	Executes the prepared model through CPU WebAssembly	Provides local inference in the target browser
Python HTTP server	Serves the application and prepared assets on 127.0.0.1	A small standard-library server is sufficient for local use
Python and pytest	Define checks, prepare fixtures and report assertions	Keep the core automation in one language with a mature test runner
Selenium	Operates an isolated Chrome session and reads its page	Exercises the user-visible browser behavior

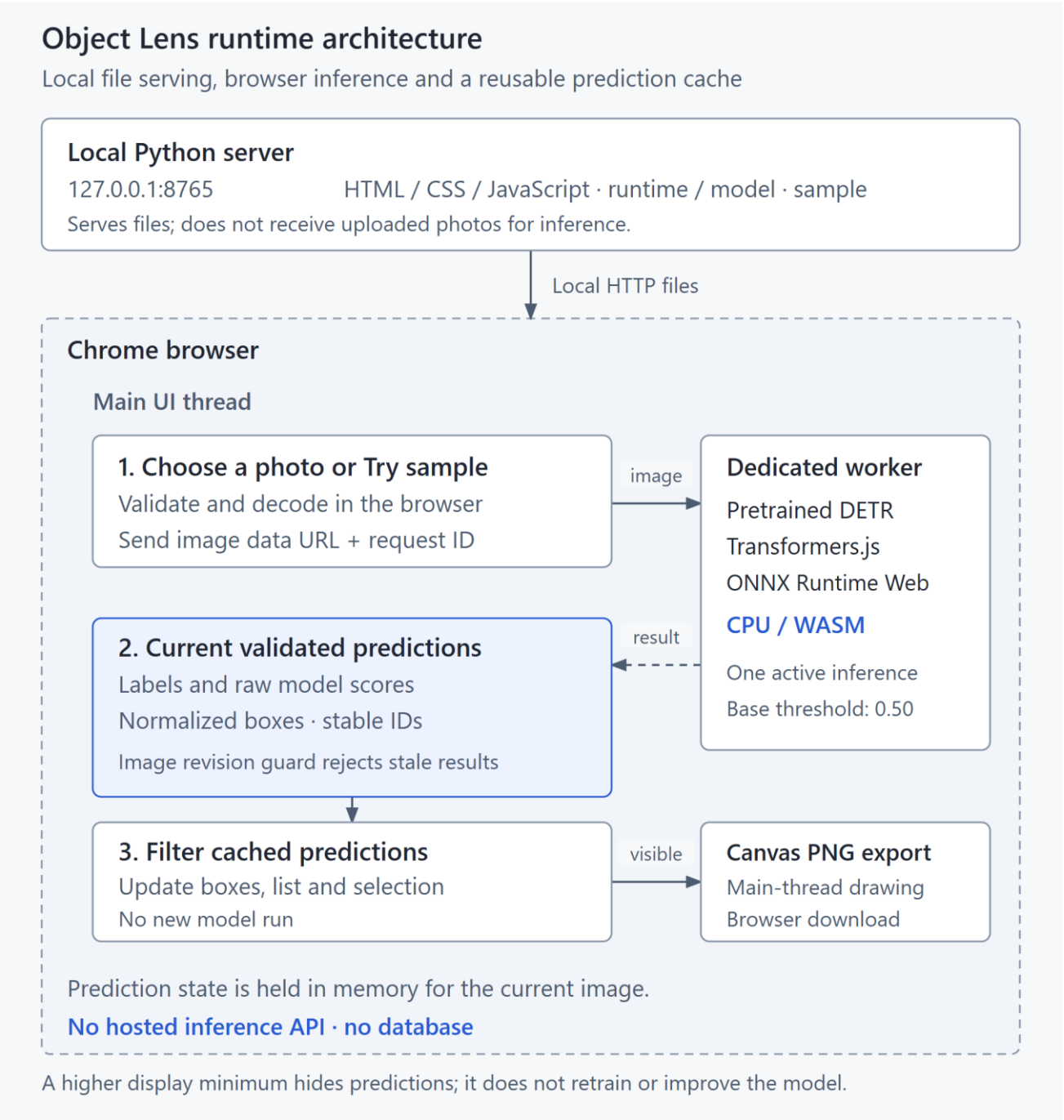


Figure 2 Runtime responsibilities. The local server provides files; the browser performs inference and draws PNG output. The separate automated-testing workflow appears in Figure 5.

The prepared application uses local runtime and model files. The worker sets remote-model access off and requests the pinned local model. An initial setup still needs downloads for dependencies, model assets, public samples and potentially the matching Chrome driver. Local inference therefore means that the prepared inference path does not call a hosted prediction service.

The recorded base uses Transformers.js 2.17.2 and ONNX Runtime Web 1.14.0 with a quantized Xenova/detr-resnet-50 model. Exact sources, model revision, file sizes and SHA-256 hashes are recorded in asset-lock.json. The version pins make the tested setup reproducible; they do not mean that these are the newest releases.

Server and worker boundaries

The Python server binds to 127.0.0.1, which addresses this computer. It is not an online demo service. The browser worker owns model loading and inference. The page owns the chosen image, the current request identity and everything displayed to the user.

Running inference in a worker keeps model execution out of the main page thread, but it does not make the computation free or provide a speed guarantee. This version uses CPU/WASM with one thread. It makes no GPU, NPU or semiconductor validation claim.

Back to contents

Image processing and result behavior

From file bytes to predictions

When a person chooses a file, the application checks the allowed format, file size, successful decoding and image dimensions. A filename extension alone is insufficient: a file called photo.png can contain unreadable or unsupported bytes. Validation prevents unsuitable input from reaching the expensive model call.

The real worker passes the image to the object-detection pipeline. The model uses learned parameters from its earlier training to compute predictions from the image pixels. The application does not select a predefined answer from the filename, and uploading an image does not train or update the model.

Excerpt 1 Real inference call from the final worker

```
const predictions = await detector(data.image, { threshold: 0.5, percentage: true });
```

Source: dist/worker.js: 42

Table 3 One prediction

Field	Meaning	How the interface uses it
label	A supported class name predicted by the model	Text in the result row and box label
score	The model's score for that prediction	Displayed beside the label and compared with the display minimum
box	Normalized xmin, ymin, xmax and ymax coordinates	Position and size of the box relative to the decoded image
Stable prediction identity	An application identity retained for the current result set	Keeps selection and color associated with the same prediction when filtering

A score is not a measured accuracy percentage. Two predictions can describe the same visible object, and a high-scoring prediction can still be incorrect. The interface therefore counts predictions rather than claiming a verified count of physical objects.

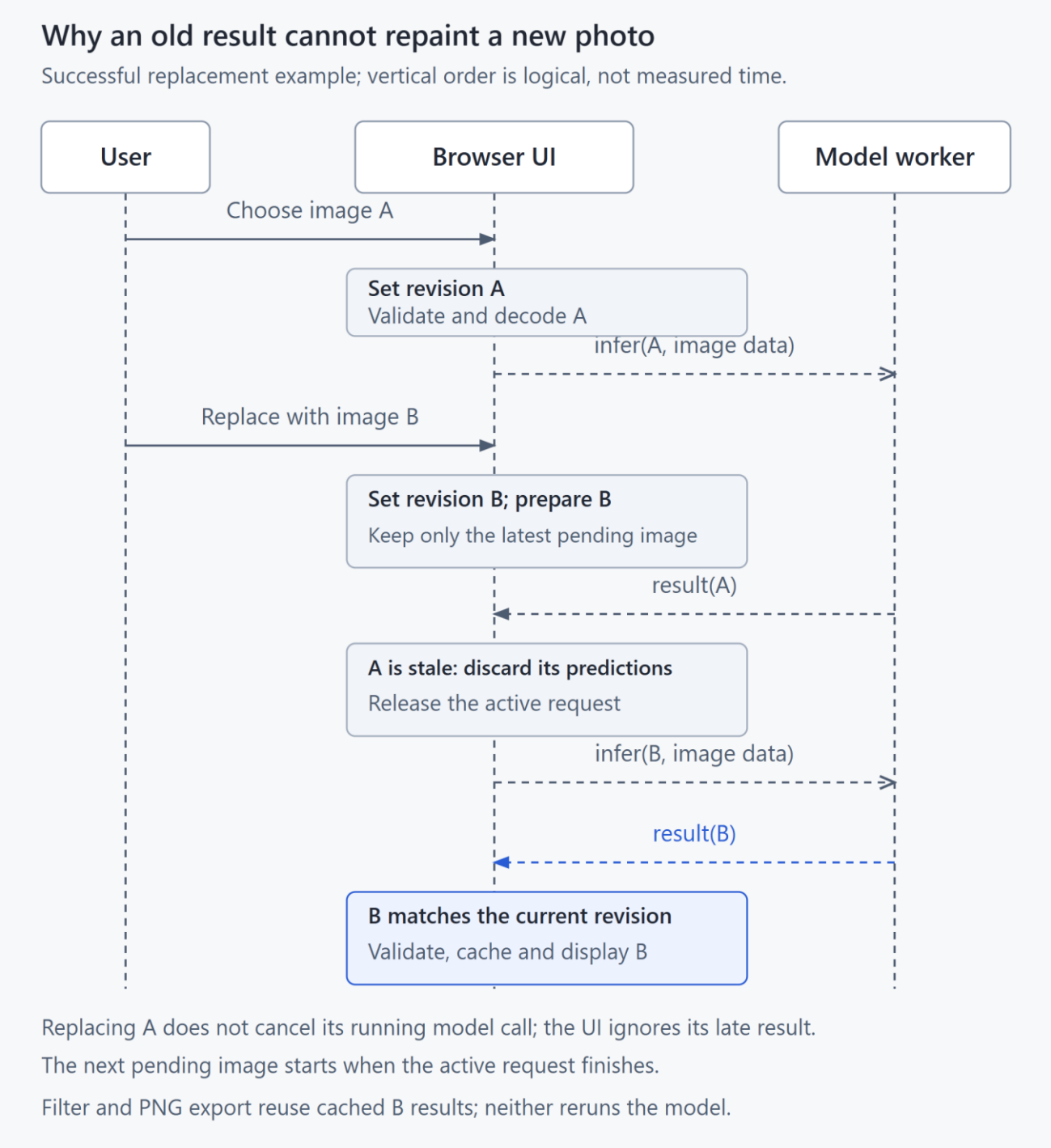


Figure 3 Image A finishes after the user selects image B. The application discards A's stale reply, then processes the latest pending image. Filtering and PNG export use retained results after inference.

Current image ownership

Every new image workflow has a revision identity. A result is accepted only when its request belongs to the current image. If an earlier image finishes after replacement or Clear, its reply cannot repaint the new page. This is an important distinction: discarding an old reply does not promise that the CPU work was interrupted.

The worker handles one active request, while the page retains at most the latest pending image. This prevents rapid replacements from turning into an unbounded list of obsolete requests.

Bounded recovery

Model loading has a 120-second watchdog and inference a 90-second watchdog in normal operation. A nonresponding worker is terminated, the error is explained, and the user can retry. Controlled tests use a shorter four-second deadline to exercise this recovery path.

An empty prediction set, an invalid image and an inference failure have different meanings. Empty results mean the detector returned no predictions under its configured output threshold. Invalid input failed validation. An inference failure means the requested processing did not complete successfully. The page should communicate the correct state and offer the applicable recovery.

Back to contents

Filtering samples and PNG output

Minimum model score

The display filter ranges from 0.50 to 1.00 in steps of 0.01. It starts at 0.50 for a newly chosen or sample image and resets on Clear. The filter compares the unrounded model score with the selected minimum and includes a prediction when its score is greater than or equal to that value.

The real detector is still requested with its existing 0.5 output threshold. Raising the display minimum hides predictions already returned by that call; lowering it restores retained predictions down to the available 0.50 floor. The filter cannot recover predictions the detector did not return, and it does not change the model's parameters.

In the recorded base coffee run, one spoon prediction scored about 0.67 and another about 0.93. A display minimum of 0.80 would hide the first and retain the second. This is a concrete filtering example, not evidence that the retained prediction is necessarily correct.

The boxes, list and visible count update from the same filtered set. Predictions retain their original identities and colors. If the selected prediction remains visible, selection is preserved. If it is hidden, the first remaining prediction becomes selected; if none remains, selection is cleared.

Source: dist/app.js:391-481; tests/test_filter.py:86-149

When the model returned predictions but the filter hides all of them, the page says that no predictions meet the minimum score and invites the user to lower it. The image stays visible and the slider stays enabled. If the detector returned an empty set, the page uses a different empty message. Displayed scores are rounded to two decimals; comparisons use the original score.

Source: dist/app.js:409-480; tests/test_filter.py:108-124

Try sample

Try sample loads the existing public coffee photo through the ordinary input and real-inference flow. Its purpose is to make the application easy to try without browsing through folders. The photo is a genuine input, not a shortcut to a fixed prediction list.

The sample file and its attribution already belong to the prepared local assets. A missing or unreadable sample produces an actionable error and a new Try sample attempt can recover. A 15-second deadline aborts a fetch that does not finish.

Source: dist/app.js:275-307; tests/test_sample.py:102-118,168-193

Sample loading reserves a new image revision before starting its fetch. Clear or a newer image selection aborts that request and retires its identity. A late response cannot become the current image.

Controlled delayed-response tests exercise both Clear and replacement; this is verified prevention of a design risk, not a claim that a production incident occurred.

Source: dist/app.js:264-307; tests/test_sample.py:120-148

Save result

Save result creates a PNG of the decoded, correctly oriented image at its original pixel resolution, with only the currently visible prediction boxes, labels and model scores. It includes no summary strip. The source image file is not overwritten.

The on-screen boxes are HTML elements over the image. PNG export draws the same visible prediction data onto a canvas in the page's main thread. Coordinates are scaled to the decoded image dimensions, and text labels include the class and rounded model score. Label backgrounds and text are kept within the canvas. The saved PNG is therefore checked separately from the screen.

Source: dist/png-export.js:3-61; tests/test_export.py:86-133

Export captures one image, one visible prediction set and the selected minimum. Clear, replacement or a filter change invalidates an export still being encoded, so it cannot initiate a stale download. Save is disabled when no predictions are visible and while an export is being prepared. An encoder failure or the 15-second encoding deadline leaves the displayed result intact and makes Save available for another attempt.

Source: dist/app.js:43-98; dist/png-export.js:44-60; tests/test_export.py:137-246

The application reports PNG download started after handing the generated file to the browser. It does not claim that the operating system has confirmed a completed save. The filename includes the source image stem and the display minimum, for example coffee-object-lens-min-0.80.png.

Source: dist/app.js:61-98; dist/png-export.js:63-67

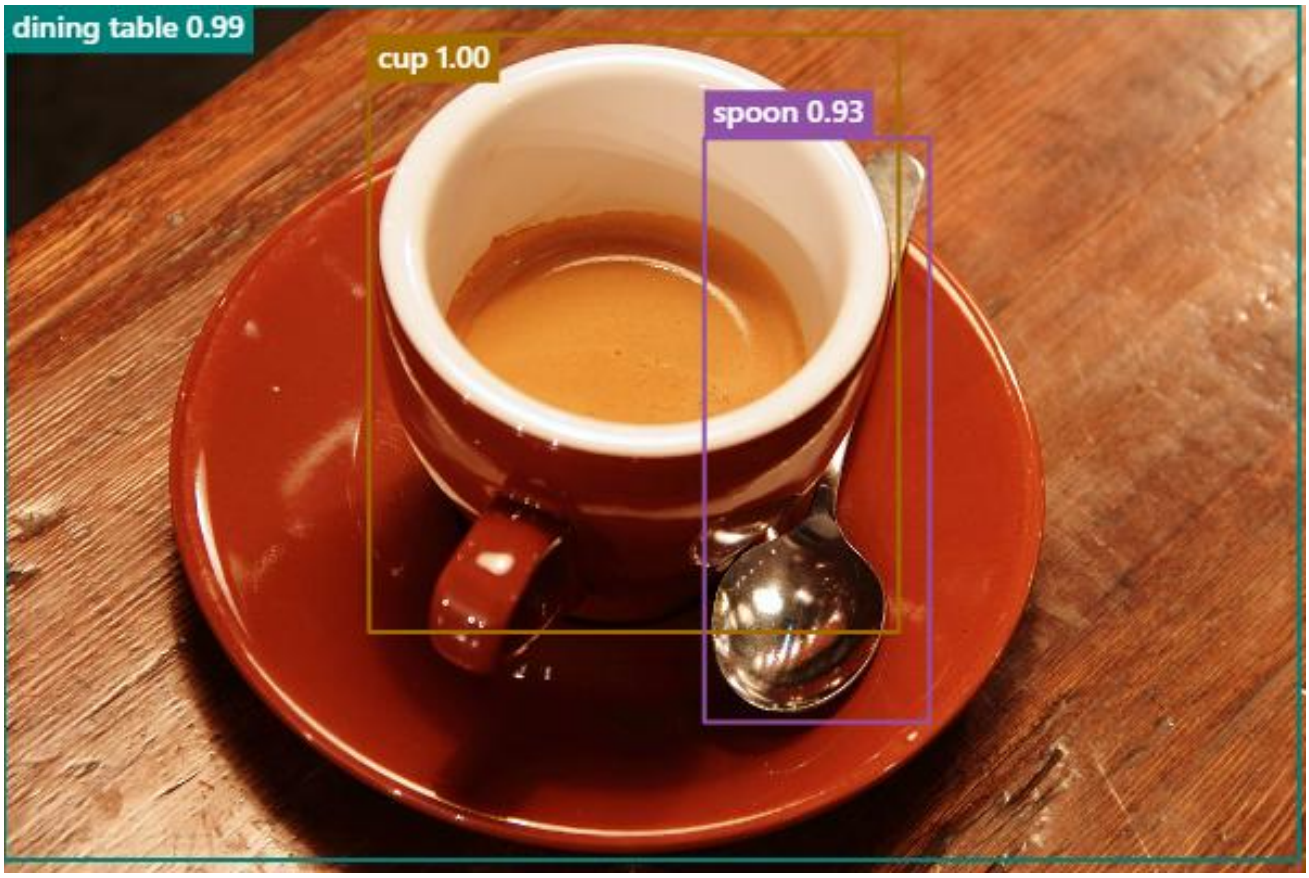


Figure 4 Actual PNG saved from the coffee sample at minimum score 0.80. It contains the photo and three visible prediction annotations, without a summary strip.

The saved image contains model predictions. A recipient should still review the photo and interpret the labels as predictions rather than verified facts.

Back to contents

Testing with Python pytest and Selenium

The two commands

Table 4 Application and test commands

Command from the project folder	What it does	Why to use it
.\.venv\Scripts\python.exe serve.py	Starts the local application server	Use or demonstrate the application manually
.\.venv\Scripts\python.exe -m pytest	Runs the written automated tests	Check whether application behavior matches the expected rules

The server command stays running until it is stopped. The pytest command finishes after the test functions and their cleanup. Tests start their own temporary servers and headless Chrome sessions, so the manually started app is not a prerequisite.

Python supplies the language in which the tests are written. pytest discovers test files, arranges fixtures, executes test functions and reports outcomes. Selenium follows the actions described by a test: selecting a file, clicking a control and reading the displayed page. Assertions compare the observed values with the expected rule.

Controlled checks and actual inference

The controlled test worker deliberately supplies known predictions, delays and failures. This lets a test reliably exercise an empty set, a timeout or a stale response. It does not establish how accurately the real model recognizes objects. Normal application mode uses the real worker; controlled mode must be enabled explicitly and is visibly labelled.

Tests marked inference run the actual prepared model. They establish that the integration can execute and record the output on the chosen sample images. A few public photos are not a labelled accuracy benchmark.

Run a focused group

```
# Controlled application and server checks
.\.venv\Scripts\python.exe -m pytest -m "not inference"

# Actual-model checks
.\.venv\Scripts\python.exe -m pytest -m inference
```

In the second command, Python interprets the first -m as run this installed module. pytest interprets the later -m as select tests with this marker. The identical spelling belongs to two different command-line interfaces.

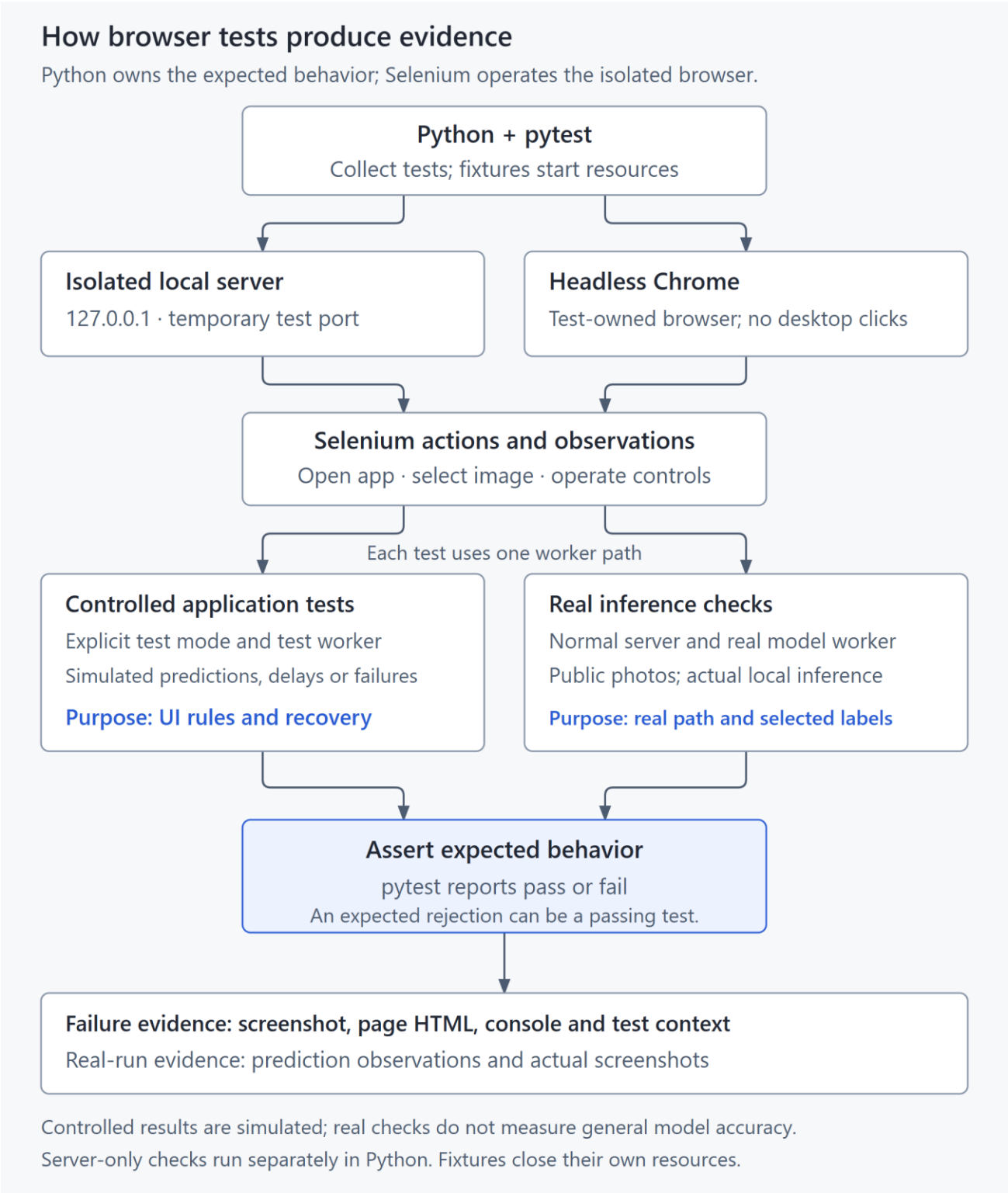


Figure 5 pytest runs written checks through Selenium. Controlled worker responses make interface scenarios repeatable; separately marked checks exercise the actual local model.

A worked requirement

The maximum file size is inclusive: a valid file exactly 10 MiB must be accepted, while a file one byte larger must be rejected. This is a boundary test because defects often occur at the point where accepted input becomes rejected input. A test far below the limit would not establish the inclusive rule.

Excerpt 2 Representative boundary test from the final implementation

```
@pytest.mark.parametrize("size,expected_state", [(10 * 1024 * 1024, "complete"), (10 * 1024 *
1024 + 1, "error")])
def test_file_size_boundary(app, make_image, size, expected_state):
    upload(app, make_image("size-boundary.png", total_bytes=size))
    wait_state(app, expected_state, timeout=20)
    if expected_state == "error":
        assert app.find_element(By.ID, "error-message").is_displayed()
        assert result_labels(app) == []
    else:
        assert result_labels(app) == ["laptop", "cup"]
```

Source: tests/test_ui.py: 223-231

The decorator runs the same test at two sizes: exactly 10 MiB and one byte above it. Read the function in four parts: prepare a valid image with the chosen byte count; submit it through Selenium; wait for an observable state; assert the expected rejection or controlled prediction list. The image must remain decodable so the case reaches the intended size rule. The laptop/cup labels are deliberate test-worker responses, not recognition results.

A negative test can pass when the application displays an error. For an oversized file, rejection with a useful message is the required behavior. pytest reports failure when the observed behavior differs from that requirement; it does not independently decide what the product ought to do.

Back to contents

Requirements and test design

Acceptance criteria describe externally observable behavior. The test plan chooses inputs and actions that could reveal a violation. Mapping a test to a requirement makes a failure interpretable and exposes requirements that have only manual or limited evidence.

Table 5 Acceptance criteria and check locations

ID	Acceptance criterion	Representative checks
AC1	Accept valid JPEG/PNG within byte and pixel limits; reject invalid input usefully	test_ui.py: test_file_size_boundary, pixel boundary, invalid/truncated content and JPEG orientation
AC2	The current image has one matching row for each displayed box	test_ui.py: current-image row and box matching; test_inference.py: actual-model observations
AC3	Selection highlights its matching box; Clear removes current content	test_ui.py: result selection, replacement and keyboard Clear
AC4	Older results cannot repaint a replacement image or cleared view	test_ui.py: delayed result replacement and Clear; test_sample.py: late fetch after Clear/upload
AC5	Invalid input, empty results and inference failure remain distinct and recoverable	test_ui.py: invalid input, controlled empty/error, startup failure and nonresponding worker
AC6	Keyboard controls and narrow layouts remain usable	test_ui.py: clientWidth overflow check at 320/390; test_filter.py and test_sample.py: keyboard actions

ID	Acceptance criterion	Representative checks
AC7	The filter consistently selects retained current predictions using raw scores	test_filter.py: six cases for exact score, restoration/no new inference, identity/selection, reset, errors and keyboard
AC8	Try sample follows ordinary image validation and inference	test_sample.py: seven cases; test_inference.py: real coffee through Try sample
AC9	PNG output contains the current visible annotations at oriented resolution	test_export.py: eight cases, including orientation, retained/excluded pixels, failed/late encoding and recovery

Source: tests/test_ui.py; tests/test_filter.py; tests/test_sample.py; tests/test_export.py; tests/test_inference.py

The check locations identify assertions in the documented source. The execution record in Verification and release assessment identifies which checks actually ran. Controlled fixture passes establish the specified application behavior, while actual-model runs supply a different and limited form of evidence.

Choosing useful cases

The score-filter checks include 0.7999, exactly 0.80 and 0.95 against a minimum of 0.80. The first prediction is excluded even though its displayed rounded score is 0.80; the other two remain. A message counter verifies that filtering and restoring predictions do not trigger a second inference. Separate cases check stable identity, color, selection, an all-hidden state and new-image reset.

The PNG checks verify decoded dimensions, including a JPEG stored at 320 by 240 pixels that displays and exports at 240 by 320 after EXIF rotation. Known pixel locations check for a retained box, an excluded box and an unchanged background. Encoder-failure and delayed-callback cases test recovery and cancellation. The actual-model coffee export separately demonstrates the full integration.

Source: tests/test_export.py:86-246

Exploratory testing complements the scripted cases. A short charter is to change input while the app is busy, adjust the filter with the keyboard and inspect whether every visible result still belongs to the displayed image. Record the inputs and observations so an unexpected result can be reproduced.

A long repeated-use session checks a different risk from a single functional test. It can reveal observed state or recovery failures during that session, but it does not certify memory-leak freedom, production availability or concurrent-user scalability.

The export-message element remains present as a polite live region even when empty; styling adds spacing only when it has text. This preserves the intended announcement mechanism. DOM/CSS inspection, keyboard checks and visible focus do not certify how every screen reader will announce it.

Source: dist/index.html:82; dist/style.css:192-194

Back to contents

Failure investigation

A useful failure report states the expected rule, the exact steps and input, the observed behavior and the evidence needed to reproduce it. Start with what happened before proposing a cause. A screenshot shows the visible symptom; browser logs, test assertions and request identities can explain the path that produced it.

Table 6 Evidence to retain

Evidence	Question it helps answer
Test name, assertion and command	Which expected rule disagreed with the observed state
Input and controlled scenario	Whether another run can repeat the same conditions
Screenshot and page HTML	What the page actually displayed at the failure
Browser console output	Whether runtime or worker errors accompanied the symptom
Source version and environment	Which implementation and browser/runtime were exercised
Follow-up result	Whether the proposed fix addressed the original reproduction

A real narrow viewport defect

The extension review found a genuine layout defect at a 320 CSS-pixel viewport. A vertical scrollbar left 305 pixels available to the document, but the body still had a 320-pixel minimum width. The page overflowed horizontally. The preserved failing test reports the comparison 320 <= 305 + 1 as false.

Narrow viewport before and after the correction

Measurement	Before	After
Requested CSS viewport	320 px	320 px
Available document width	305 px	305 px
Document content width	320 px	305 px
Overflow beyond available width	15 px	0 px

The earlier check compared content width with window.innerWidth, which included space unavailable to the content when the scrollbar was present. It could therefore pass while the page still overflowed. The corrected check sets the CSS viewport explicitly and compares scrollWidth with documentElement.clientWidth, allowing one pixel for rounding. It also checks that each essential control fits within the available width.

Changing the body's min-width from 320px to 0 lets the responsive layout use the actual available width. A subsequent real-model capture measured content and available widths of 305 and 305 at 320, and 375 and 375 at 390. The 768- and 1440-pixel layouts also fit their available widths. This finding combines an application fix with a stronger test measurement; it was not an injected source fault.

Source: docs/evidence/viewport-before-fix.xml; dist/style.css:16; tests/test_ui.py:320-342; docs/evidence/extension-demo.json

A deliberate stale-result fault

The base verification included an isolated experiment that disabled the current-image revision guard. A delayed result for an earlier image could then appear on a replacement image, and the corresponding test detected it. This demonstrates that the check can expose that chosen fault. The fault was deliberately introduced in an isolated copy; it was not a naturally discovered production incident.

The investigation connects the symptom to a narrow mechanism: an old request returned after the displayed image changed, and the altered copy no longer rejected it. Restoring the guard addresses

that mechanism. This is first-level application investigation, not a claim about a defect inside the pretrained model.

A prediction mistake is a different kind of finding

The real base coffee run included duplicate spoon predictions, while another sample produced incorrect and duplicate labels. Those observations describe model behavior. If the application accurately renders the returned data, an incorrect model label is not by itself proof that the rendering code is broken.

The response should match the finding: fix a reproducible application defect, improve the wording if the UI overstates a prediction, or record a model limitation. Raising a score filter may hide some predictions but does not establish general accuracy improvement.

An unexpected file in a download check

The first combined extension run recorded 47 passes and one failure. The failure occurred when a timeout test expected an empty download folder but found downloads.htm.crdownload. The retained investigation identified a non-PNG CRX3 container holding model data. The producing process or component was not established, so the finding does not prove that Object Lens generated that file.

Source: docs/evidence/extension-first-full-suite.xml; docs/evidence/extension-verification.json

The strict assertion was retained. The isolated test browser was then configured to resolve only localhost and 127.0.0.1 because its application and model assets are served locally. This change is confined to the test browser and does not alter the computer's network or security settings. The subsequent complete rerun passed all 48 cases.

Source: tests/conftest.py:81-88; docs/evidence/extension-full-suite.xml

Back to contents

Verification and release assessment

The following checkpoints were executed on 2 November 2026. Their case counts overlap, so they must not be added together. The final combined suite is the regression result for the integrated extension. Historical base evidence is identified separately.

Source: docs/evidence/score-filter-record.json; docs/evidence/sample-image-record.json; docs/evidence/export-png-suite.xml

Table 7 Execution record and evidence limits

Check	Result	Evidence and limit
Final combined suite	48 passed; 0 failures/errors/skips; 418.62 s	45 controlled application/server cases and three actual-model cases; extension-full-suite.xml
Filter checkpoint	29 passed; 199.37 s	23 existing controlled checks and six filter cases; actual-model cases deselected
Sample checkpoint	10 passed; 137.27 s	Seven controlled sample cases plus three actual-model cases
PNG checkpoint	8 passed; 89.03 s	Actual PNG encoding/download with controlled predictions and deliberate encoding faults

Check	Result	Evidence and limit
Real feature demonstration	Passed on the coffee sample	Four original predictions; three at 0.80; actual 600 x 400 PNG, no captured console entries or external HTTP requests
Rendered viewport review	320, 390, 768 and 1440 CSS px fit	Content width compared with available clientWidth; keyboard/accessibility limits stated separately
Earlier base long session	320 cycles in 3615.927 s	Historical five-photo repeated-use observation; does not cover the extension

The separate real-model demonstration captured no page HTTP requests to external hosts and no browser-console entries. Those observations describe that browser session; they are not a measurement of all operating-system network activity. Selected JUnit records remove machine hostname attributes, and the evidence manifest records the raw-source hashes and that redaction.

Source: docs/evidence/extension-verification.json; docs/evidence/extension-demo.json

The recorded environment uses Python 3.12.10, pytest 9.0.3, Selenium 4.40.0, Chrome 156.0.8078.74 and ChromeDriver 156.0.8078.80 on one Windows laptop. Pinned asset versions and checksums remain in asset-lock.json. This is a local execution record, not a second-machine or cross-browser qualification.

Source: docs/VERIFICATION.md; docs/evidence/extension-demo.json

The earlier base also recorded a complete suite with 26 passes, comprising 23 controlled application/server checks and three actual-model checks. Those earlier results describe the base version. They are not added to the extension counts or used as proof that the new filter, sample or PNG behavior passed.

The sample expectations were based on upstream image descriptions and separate AI-assisted visual review before inference. They are not exhaustive human annotations. The project has not established a general detection-accuracy figure, production performance, Safari or iOS behavior, or semiconductor defect-detection capability.

Release decision

Release for the agreed local learning and demonstration workflow is recommended for the documented version. Final complete-suite result: 48 passed; 0 failures, errors or skips; 418.62 seconds. The focused feature checks, actual-model sample demonstration, downloaded PNG inspection and corrected viewport measurements provide the supporting evidence. The release remains limited to the tested local environment and the specified acceptance criteria.

Source: docs/evidence/extension-verification.json; docs/evidence/extension-full-suite.xml; docs/evidence/extension-demo.json

A private local release means the documented version is suitable for its agreed local use under the tested conditions. It does not mean that the source has been published, that an online service is deployed, or that an employer has accepted the project as proof of role qualification.

Back to contents

Running and maintaining the project

Use the application

Start the local server

```
Set-Location 'C:\Users\lejtan\Downloads\Apple-SDET\object-lens'
.\.venv\Scripts\python.exe serve.py
```

Open http://127.0.0.1:8765 in Chrome and keep the terminal running. Ctrl+C stops the server. If the project is moved, adjust the Set-Location path. If that port is already in use, choose another with serve.py --port 8766 and open the corresponding local address.

For a short demonstration, use Try sample, wait for the real predictions, select a result, raise and lower the minimum score, then save the visible annotations. Open the resulting PNG separately and compare it with the current view. Finally choose a different photo or Clear.

Run checks

Execute the suite

```
.\.venv\Scripts\python.exe -m pytest
```

The tests own their temporary servers and headless browsers. They should clean up those resources when finished. A manually opened app can remain available in another terminal; do not confuse its state with the isolated test browser.

Table 8 Where to look

Location	Purpose
dist	The application and prepared local runtime/model/sample assets
tests	Python acceptance checks, server checks and browser fixtures
docs/DESIGN.md	Concise design and acceptance criteria
docs/VERIFICATION.md and docs/evidence	Recorded results and selected evidence
tools and asset-lock.json	Preparation helpers and exact asset sources/hashes
docs/system-design/source.json	Authoritative prose, tables and captions for this companion
docs/system-design/diagrams	Editable diagrams and their SVG/PNG exports
.venv, .local and artifacts	Generated environments and local working or raw-evidence files

Make a bounded change

Change one behavior with its acceptance criterion and relevant tests. Run the focused checks while developing, then the applicable wider suite and UI review before committing. A test that simply repeats the implementation's calculation can reproduce the same defect, so expected outcomes should follow the requirement or an independent example.

Keep the app's image identity, visible predictions and exported output consistent when changing result behavior. Keep real inference separate from controlled fixtures. Preserve existing attribution and the recorded asset sources when updating dependencies.

Update source.json and the relevant diagram source when the implementation changes, then rebuild the Word document and its matching PDF. The contents use internal hyperlinks and page numbers verified against the rendered document. Recheck those targets and every final page after changes. Editing a generated Word or PDF copy alone does not update the authoritative source.

AI assistance and honest demonstration

AI tools assisted with implementation, test authoring and documentation. The app adapts an upstream object-detection example and uses an existing pretrained model. It does not claim that this project trained the detector or that every automated check was independently written by the owner.

A demonstration can honestly describe the application workflow, the role of each technology, one acceptance criterion and its test, an investigated failure, and the scope of the release decision. The owner's understanding should be demonstrated through explanations and actions actually performed, without inventing prior experience.

Back to contents

Terms and source references

Table 9 Terms in this application

Term	Meaning
Inference	Running a trained model on an input to produce predictions
Pretrained model	A model whose learned parameters were produced before this application used it
Model score	The number the model assigns to a prediction; not a measured accuracy percentage
Threshold	A selected minimum used to include or exclude predictions
Bounding box	A rectangle describing the predicted location of an object
WebAssembly	A browser execution format used by this runtime to run model operations on the CPU
Quantization	Using a lower-precision model representation; this application selects a prepared quantized model instead of training one
Web Worker	A browser execution context separate from the page's main thread
Fixture	Reusable test setup, inputs or controlled conditions
Assertion	A check comparing an observation with an expected rule
Regression	Previously correct behavior becoming incorrect after a change
Stale response	A result belonging to an earlier image or operation
Acceptance criterion	A concrete observable condition the implemented feature must satisfy

Term	Meaning
Release decision	A conclusion about readiness for a stated use, based on evidence and known limits

Application behavior is established by the documented source and its retained execution records. Upstream documentation explains library concepts and APIs; it is not evidence that this application passed a check.

Table 10 Source index

Subject	Source
Approved scope and acceptance criteria	docs/DESIGN.md
User workflow and commands	README.md and docs/LEARNING.md
Application state and rendering	dist/app.js, dist/png-export.js, dist/index.html and dist/style.css
Actual model execution	dist/worker.js and asset-lock.json
Test setup and behavior	tests/conftest.py; test_ui.py; test_filter.py; test_sample.py; test_export.py; test_inference.py; test_server.py
Verification records	docs/VERIFICATION.md, docs/evidence and retained local artifacts
Public sample provenance	docs/IMAGE_SOURCES.md
Upstream example and attribution	NOTICE.md and baseline/source.json
Transformers.js pipeline documentation	Transformers.js 2.17.2 pipeline API
Selenium documentation	Selenium browser automation
pytest documentation	pytest test runner

Back to contents